Open in app ↗ Sign up Sign in

Medium Search

Android + Nfc + Host Based Card Emulation + Nfc Reader + Programming +

How to emulate a Credit Card on Android with Host Based Card Emulation (“HCE”) in Java

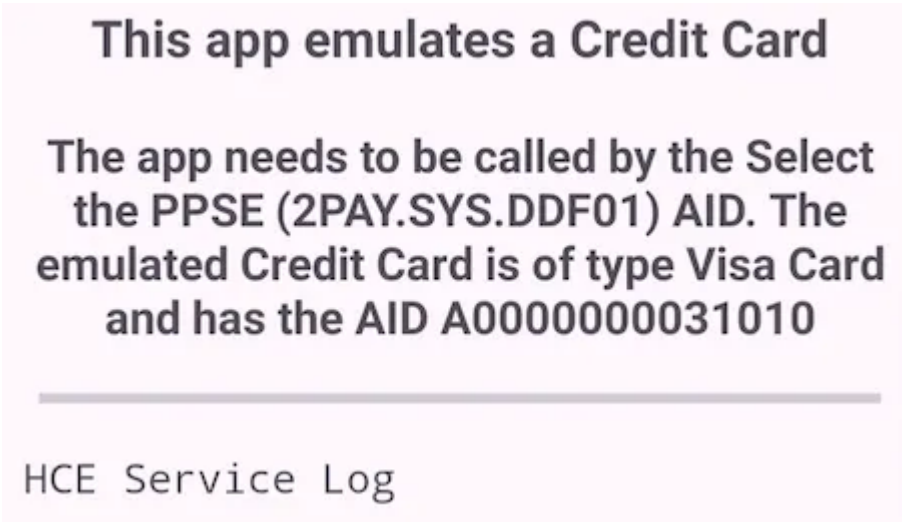
26 min read · Oct 5, 2024



AndroidCrypto Follow

Listen Share

This app will explain how to run an **emulation of a Credit Card** (“EMV card”, CC) using the **Host Based Card Emulation (HCE)** feature on Android devices. It is showing the basics on how to get called by a Credit Card Terminal (“POS Terminal”, an NFC Reader), process the incoming messages from the terminal and respond accordingly to the terminal.



The complete emulation process is a kind of **“Question & Answer” game** — the terminal requests some data, the (emulated) card responds the requested data, the terminal requests new data (based on the data send
to the terminal in the first step) — the card is answering and so on. It may take up to

10 rounds before a transaction is finished and processed by the CC terminal (“POS Terminal”).

The complete source code is available in a **GitHub repository**, you will find the link to the repository at the end of this article. This tutorial is based on an article I published here on medium.com — **How to use Host-based Card Emulation (HCE) in Android — a beginner tutorial (Java)**. I strongly recommend to read this tutorial before starting with this one as I won’t repeat some parts here.

The second recommendation is for a “**POS Terminal Emulator**” that I wrote for another tutorial. As one Android device is going to emulate a Credit Card you need “the other side” to test the card. If you are starting with HCE Emulation you won’t use a professional card reader, because usually you get just two kind of responds from those terminals — “accepted” or “declined”. My application gives you a log file to monitor what is requested by the reader and what data is responded by the emulated tag. The complete app is accompanied by an article series “**Talk to your Credit Card**” that gives you a very deep understanding what data is stored on a Credit Card and the how the data is organized. A “ready to use” compiled APK is available for downloading in the **GitHub repository**.

To avoid any disappointments

This app is just the base for (your !) final app and will help you to understand how the basics of the Host Based Emulation for emulating Credit Cards is setup, but it will not be a “ready to use” application that is being able to run a complete transaction on a real POS Terminal !

A second note: for good reasons a Credit Card (EMV card) is not a simple NFC storage tag that you can read or write. Even when comparing a Credit Card with a MIFARE DESFire EV3 tag the EMV tag has some features that go far beyond the DESFire one. The most important reason is that sensitive data should never leave the tag — they will **never be exposed** by a read command. Some tasks when working with an EMV card include cryptographic calculations, they are completely done within the tag and only the result will be responded to the POS Terminal. Some operation e.g. are secured by a digital signature that is calculated by the EMV card on the base of transaction parameters. A digital signature is done by using a **Private Key** and later verified with the **Public Key**. As the Public Key is available (as the name says) in the public everyone can use it to verify the response (or better the signature based on the response). The Private Key is written to the tag during card

manufacturing by the Credit Card issuer
and you will never get knowledge about this key. So your emulated CC will never being able to create a signature that can get responded to the terminal and has a valid signature. Of course, not every transaction may get processed with high “security” priority, but in general it is really hard
to make a working code. For that reasons there are no apps on the market that emulate a Credit Card. Of course, there are solutions like “Google Wallet” or “Apple Pay” that work like a Credit Card, but in the end they are token based and may use the same card account but not the data from your physical Credit Card.

That’s enough with bad vibes, our journey begins right now.

Three steps to create a HCE Credit Card emulator

1 Setup an application with just a MainActivity class (Activity)

As the HCE Emulation is done in a **background service** we do not need much UI stuff. Additionally create a service class (e.g. HceCcEmulationServiceBasic.java”) that **extends HostApduService**. See the HCE Beginner tutorial on how to do.

2 Modify your AndroidManifest.xml file

As we need a binding of the service class to our app we need to append the HCE related service data:

```
<service
    android:name=".HceCcEmulationServiceBasic"
    android:exported="true"
    android:permission="android.permission.BIND_NFC_SERVICE">
    <intent-filter>
        <action android:name="android.nfc.cardemulation.action.HOST_APDU_SERVIC
        <!-- category required!!! this was not included in official android HCE
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
    <meta-data
        android:name="android.nfc.cardemulation.host_apdu_service"
        android:resource="@xml/apduservice" />
</service>
```

There are two file names defined: the “*android:name=”.HceCcEmulationServiceBasic*” is pointing to the service class you created in 1) and the resource for meta-data:

“*android:resource=”@xml/apduservice” />”* — this is the file “*apduservice.xml*”, located in the “res” part of your project in the subfolder “xml”.

This is one of the most important settings you are going to do. The content of “apduservice.xml” may look like mine:

```
<?xml version="1.0" encoding="utf-8"?>
<host-apdu-service xmlns:android="http://schemas.android.com/apk/res/android"
    android:description="@string/servicedesc"
    android:apduServiceBanner="@drawable/hce_260_96"
    android:requireDeviceUnlock="false">
    <aid-group android:description="@string/aiddescription"
        android:category="payment">
        <!-- select PPSE 2PAY.SYS.DDF01 -->
        <aid-filter android:name="@string/hcePpse"/>
        <aid-filter android:name="@string/aidVisaCard"/>
        <aid-filter android:name="@string/aidMastercard"/>
    </aid-group>
</host-apdu-service>
```

There are 3 **aid-filter** defined in the file. The data behind the placeholders is in the “*res/values/strings.xml*” file:

```
<!-- select PPSE 2PAY.SYS.DDF01 -->
<string name="hcePpse">325041592E5359532E4444463031</string>
<!-- visa credit or debit card -->
<string name="aidVisaCard">A0000000031010</string>
<!-- mastercard credit or debit card -->
<string name="aidMastercard">A0000000041010</string>
```

You should not change the information given here but you can append new filters. The first filter (“hcePpse”) defines a “**Select Payment Card**” (“**Paypass Payment System Environment**” = “**PPSE**” command) call that is used by all POS terminals in the world when using contactless (NFC) cards. If you leave this command out or modify it your app will never called by terminal in default configuration. The emulated tag will respond some data to inform the terminal which applications are available on the (HCE) tag.

The second (“aidVisaCard”) and third (“aidMastercard”) entries define a possible application of a Credit Card Issuer on the tag — that are the major Credit Card Companies like “Mastercard” or “Visa”. At minimum one the two AIDs need to match the data that was responded by our HCE emulated tag on the “Select Payment Card” command. As the POS terminal receives the “PPSE respond” it selects which of (possible) multiple AIDs on the tag it should use for processing a payment. This application is called by a “Select Application by AID” command. If our Android application does not include this AID the call will never get forwarded from Android’s OS to the app (or better to the service).

A note on two other parameter in this file:

- “android:category=”payment”” is defining that our service is processing a payment (the HCE beginner tutorial is using the “other” category). On my Android devices **only ONE service can act as a payment service**, so if you should use “Google Pay” or other wallet services on this device you have to select which service is active ! Don’t forget to deselect our HCE emulated CC Service or you can’t pay any longer at the gas station… See the **Troubleshooting** chapter for more details.
- “android:apduServiceBanner=”@drawable/hce_260_96"”: is pointing to an image located in your drawable subfolder with a static size of 260 * 96 pixel. It is mandatory by Google to have this image as it is displayed on the “payment” tab of NFC settings:



3 Modify your blank service class

Simply add four lines of code to your class. The “SELECT_OK_SW” defines a response that is “SUCCESS”. The print on System’s console prints out a byte array in hex encoding. After the print the method returns a “SUCCESS” to the reader.

```
public static byte[] SELECT_OK_SW = hexStringToByteArray("9000");
...
@Override
    public byte[] processCommandApdu(byte[] commandApdu, Bundle bundle) {
        System.out.println("processCommandApdu: " + bytesToHexNpe(commandApdu))
        System.out.println("processCommandApdu: " + new String(commandApdu, Sta
        return SELECT_OK_SW;
    }
```

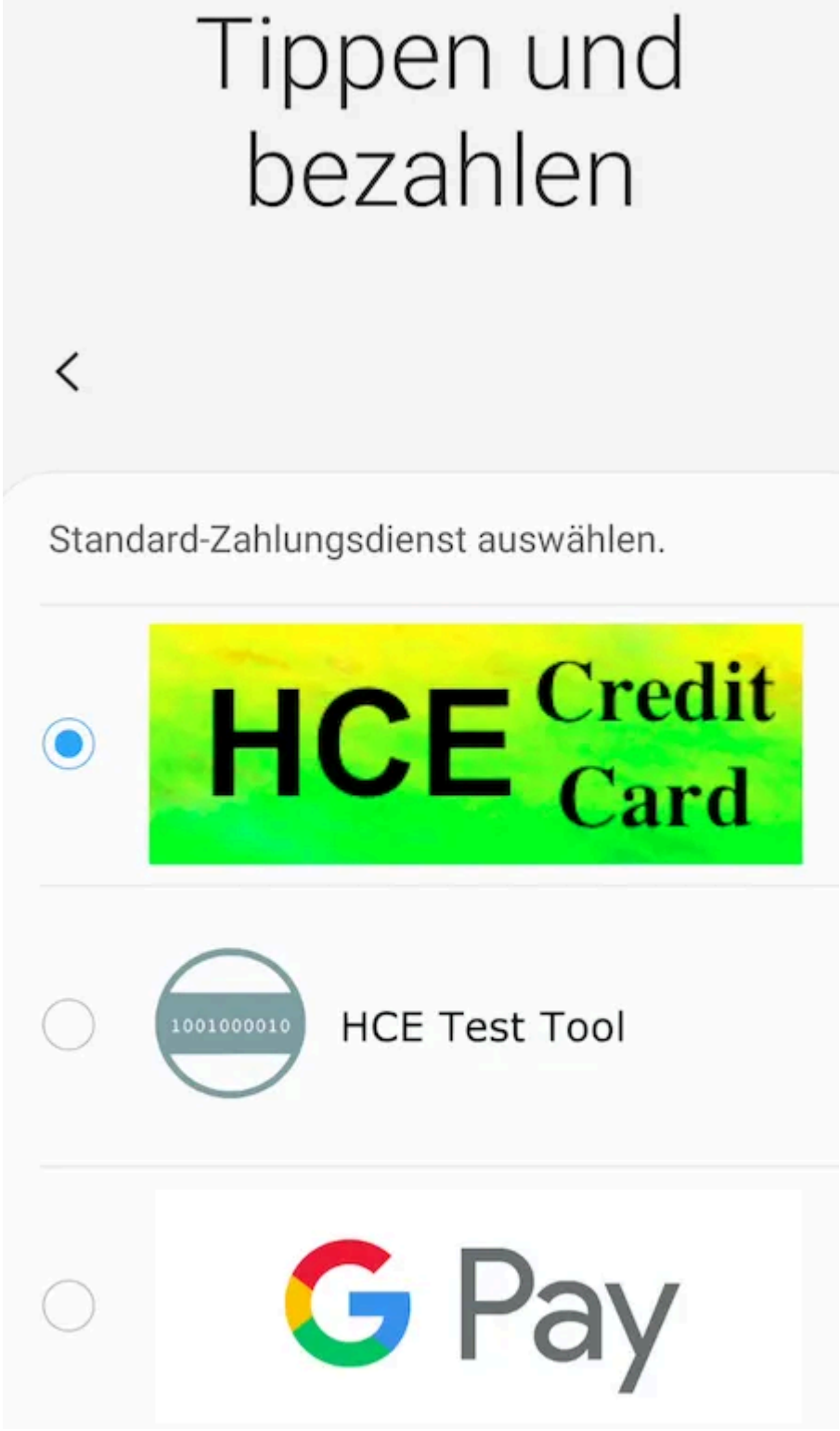
When tapping the device to the “Talk to your Credit Card” app there is just one line in the console:

```
processCommandApdu: 00a404000e325041592e5359532e444446303100
processCommandApdu: ??? ?? 2PAY.SYS.DDF01??
```

Beneath some unprintable bytes there is a cleartext message withing the incoming commandApdu: **2PAY.SYS.DDF01**. Using a search machine brings you to an article within the previous mentioned article series “**Talk to your Credit Card**” that explains what an NFC based Credit Card reader is doing to retrieve the data. In Part 1 you find the string “2PAY.SYS.DDF01” as well, it is the same as we defined in the “*apduservice.xml*” file in hex encoding.

Troubleshooting

Although you made every as described you don’t see anything in your console — how is that possible ? The answer is simple: your app is active, your service is running but the OS of your Android device does not forward any command to your service. This is due to the reason that the HCE service is for a **payment category**. There can be several HCE services running in parallel when you use the **category “other”** but there can be just **one payment application selected** that is responsible for such commands. To verify that your app is enabled to do so go to the Settings -> Connections -> NFC -> Tap and Pay -> category “payment” tab (“enable default payment service”): and enable the HCE CC app (if not already done):



Another main reason for failure is located (“invisible”) in your Android device. When developing these kind of apps you often use different service classes for testing. You can easily switch between several service classes by changing the tag in “AndroidManifest.xml” (android:name=”.HceCcEmulationServiceBasic”) to another class. But although your app will compile and re-installs the app your Android device will still use the **old service class.** The only way to change the service class is by (physically) deleting the app on your Android device and then “Run” the modified

app that installs the modified app. It is NOT necessary to delete and install the app when you are just changing the content of the service class !

Congratulations

Yes, you took the most important hurdle on your way to implement an Credit Card Emulator ! But this is not the end of our tutorial, we are going to implement 4 “question and answers” that lets your card reader start to work:

1. **Select Paypass Payment System Environment** (“Select PPSE”) command
2. **Select Application** (“Select AID”) command
3. **Get Processing Options** (GPO) command
4. **Read File(s)** command

This is just the description of the commands, an implementation will follow later.

Select Paypass Payment System Environment (“Select PPSE”) command

Any communication between an NFC reader and an NFC card is initiated by the reader. That means that the card is always responding to a previous **command** with a **response**, for that reason I’m talking about “question and answer” communication. The **Select PPSE command** is the start of communication with a payment card (“EMV” card). As the contactless payment method was started first by Mastercard and Visa they set the standard of doing this way. This is good for us as we can rely on this procedure (just one exception, see “Select Application” chapter).

As the communication has to follow the EMV standard each command contains a “**Command**” (mandatory) and in most times one or more **parameter** (optional). This is the complete byte sequence an NFC reader is sending to an EMV NFC tag (the Credit Card):

```
Select Paypass Payment System Environment command
00a404000e325041592e5359532e444446303100
00 - Header
  A4 - Command
    0400 - Sub command parameter P1 and P2
        0E - Length of following data (0x0Eh = 14 decimal)
          325041592e5359532e4444463031 - Data
                                    00 - Finalization
```

```
The Data is the hexadecimal encoded String:
2PAY.SYS.DDF01
```

Do you remember the chapter “2 Modify your AndroidManifest.xml file” ? I advised to add this data to your “aid-filter android:name=@string/hcePpse” in “apduservice.xml” file. The data for “@string/hcePpse” is defined in “strings.xml” and is “<string name=hcePpse>**325041592E5359532E4444463031**</string>”). That means 2 facts:

1. Android OS will forward a call including this AID to your app (the service) if an NFC reader is requesting so
2. The “processCommandApdu” method in “HceCcEmulationService.java” will receive this call.

Remembering the first data you received in the console ? It is exact this “Select PPSE” command. In our basic example we just responded with a byte sequence indicating an “Okay” answer, but the “Talk to your Credit Card” seems to not send any more data. This is true due to the fact that the reader awaits more information in the response. I’m starting with an example of a real, existing Visa Credit Card:

```
byte[] SELECT_PPSE_RESPONSE = hexStringToByteArray("6f38840e325041592e5359532e4
```

That are 58 bytes of data, followed by the “*0x9000h*” okay sequence. To analyze the data you can do the “hard way” by inspecting byte per byte or use an online tool from “EMV Lab” called “**TLV Utilities**” (link). This is a TLV decoder, meaning a “tag-length-value (TLV) format” decoder as the response data is encoded (mostly) in the TLV format. Copy the “SELECT_PPSE_RESPONSE” data in the form and get this decoded data (link with data):

```
6F File Control Information (FCI) Template
  84 Dedicated File (DF) Name
    325041592E5359532E4444463031
  A5 File Control Information (FCI) Proprietary Template
    BF0C File Control Information (FCI) Issuer Discretionary Data
```

```
61 Application Template
  4F Application Identifier (AID) - card
    A0000000031010
  50 Application Label
    V I S A D K B
  87 Application Priority Indicator
    01
  9F0A Unknown tag
    0001050200000000
```

Wow, this is much better than counting bytes. Well, the response is of type “**File Control Information (FCI) Template**” and informs the reader which application(s) are on the tag. In this case there is just one application available with the “Application Identifier (AID)”: “*A0000000031010*” (this is the second AID you entered in strings.xml = @string/aidVisaCard).

Please note that some data in a response can be **mandatory** (like the AID), **optional** (like the Application Label) and others are **Credit Card Issuer specific ones**, so please do not rely that all data is present on each Credit Card. Even if you may have 2 Credit Cards from the same bank the data may be different.

The reader will now check if he has a routing for the given AID(s), meaning that e.g. if the shop owner has a contract for processing Visa cards only the the POS Terminal will will not process any MasterCard or American Express cards.

Select Application (“Select AID”) command

This is usually the second command the reader is sending to the card. The AID has to be in the HCE/AID list of the application, otherwise this command is not forwarded to the app. I wrote “usually the second command”, there is one exception of that rule: if e.g. the shop owner can process just one defined AID only then the reader will skip the “Select PPSE” command and directly send the “Select AID” command.

The “Select Application” command is dynamic, meaning that the reader will select just one of the AIDs defined in the “Select PPSE” response. If your emulated Credit Card contains more then one application you need to compare the data in “Select AID” with a list of the AIDs available on your HCE tag and then — depending on the selected AID — respond the matching data.

This is how the “Select AID” looks like for our sample Visa card:

```
byte[] SELECT_AID_COMMAND = hexStringToByteArray("00a4040007a000000003101000");
```

Let’s examine the structure of the command:

```
Select Application command
00a4040007a000000003101000
00 - Header
  A4 - Command
    0400 - Sub command parameter P1 and P2
        07 - Length of following data (0x07h = 7 decimal)
          a0000000031010 - Data (here the AID)
                       00 - Finalisation

The Data is the Application Identifier (AID) that gets selected:
a0000000031010
```

What is a typical response a Visa card can give to the reader — this is again an example from me sample card and will differ to your card:

```
byte[] SELECT_AID_RESPONSE = hexStringToByteArray("6f578407a0000000031010a54c50
```

That are 89 bytes of “raw” data and I’m using the online TLV-decoder for structuring the data (link including data):

```
6F File Control Information (FCI) Template
  84 Dedicated File (DF) Name
    A0000000031010
  A5 File Control Information (FCI) Proprietary Template
    50 Application Label
      V I S A D K B
    87 Application Priority Indicator
      01
    5F2D Language Preference
```

```
    d e e n
  9F38 Processing Options Data Object List (PDOL)
    9F66049F02069F03069F1A0295055F2A029A039C019F3704
  BF0C File Control Information (FCI) Issuer Discretionary Data
    9F0A Unknown tag
      0001050200000000
    9F5A Unknown tag
      3109780276
    BF63 Unknown tag
      DF20 Unknown tag
        80
```

This is again a “File Control Information (FCI) Template” but with different content. Don’t be aware that some tag are “Unknown” as they are issuer specific ones and the general specification doesn’t know about their purpose.

The most important information the tag responds is the “**Processing Options Data Object List (PDOL)**”. The card is requesting data from terminal to proceed with the workflow (the transaction). Below you find a table what the purpose of each “tag” is(do not confuse with the NFC tag; the “tag” in this list is the data element):

```
The card is requesting this data:
9F 38 18 -- Processing Options Data Object List (PDOL)
            9F 66 04 -- Terminal Transaction Qualifiers
            9F 02 06 -- Amount, Authorised (Numeric)
            9F 03 06 -- Amount, Other (Numeric)
            9F 1A 02 -- Terminal Country Code
            95 05 -- Terminal Verification Results (TVR)
            5F 2A 02 -- Transaction Currency Code
            9A 03 -- Transaction Date
            9C 01 -- Transaction Type
            9F 37 04 -- Unpredictable Number

In total the card is requesting 33 bytes of terminal and transaction specific d
```

Please note: each tag has a length of one or two bytes length, followed by the number of bytes the returned data should have. I won’t explain all tag elements but 3 of them:

- tag “**9F 02** 06”: is the price of the goods you are going to buy in the shop, the length is 6 bytes
- tag “**9C** 01”: is the type of transaction (meaning the goods). Sometimes the provision for the shop owner differs on the type of goods, e.g. buying gas may a lower provision than buying snacks. This information is just one byte long
- tag “**9F 37 04**”: is a number that may be random so anyone who is sniffing the data cannot know which number is asked in this transaction. The length of the responded data (the number) has to be of **4 bytes**.

Note: not every Credit Card is requesting a “Processing Options Data Object List” (PDOL), for e.g. my MasterCards work without such a request. In this case the following command is “empty” regarding the PDOL data (for more details see the MasterCard chapter).

My “Talk to your Credit Card” app is using these “semi static” data for processing:

```
Tag  Tag Name                          Length Value
-----------------------------------------------------
9f66 Terminal Transaction Qualifiers     4  27 00 00 00
9f02 Amount, Authorised (Numeric)        6  00 00 00 00 10 00
9f03 Amount, Other (Numeric)             6  00 00 00 00 00 00
9f1a Terminal Country Code               2  09 78
95   Terminal Verification Results (TVR) 5  00 00 00 00 00
5f2a Transaction Currency Code           2  09 78
9a   Transaction Date                    3  23 03 01
9c   Transaction Type                    1  00
9f37 Unpredictable Number                4  38 39 30 31
-----------------------------------------------------
```

Now the NFC reader (the “POS terminal”) is asked to provide the requested data:

Get Processing Options (GPO) command

This is the third step in the workflow and — due to the nature of the command — a dynamic one, meaning you cannot identify the command by static comparing. But let’s see the data and structuring:

```
byte[] GET_PROCESSING_OPTONS_COMMAND = hexStringToByteArray("80a800002383212700
Get Processing Options (GPO) command
80a8000023832127000000000000001000000000000000097800000000009782303010038393030
80 - Command
  A8 - Instruction Code
    0000 - Sub command parameter P1 and P2
        23 - Length of following data (0x23h = 35 decimal) including '83' and '
          81 - static identifier indicating that PDOL data are following
            21 Length of the PDOL data (0x21h = 33 decimal)
              27000000 - Terminal Transaction Qualifiers
                      000000001000 - Amount of the transaction
...
                  ...     00 - Finalisation

The "Data" is each requested tag in the same sequence and length as requested
```

If you compare the data with the “semi static” data you see they are equal and in the same sequence and length as requested in the PDOL.

Get AndroidCrypto’s stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Now our tag is being able to process the transaction internally and provide this response:

```
byte[] GET_PROCESSING_OPTONS_RESPONSE = hexStringToByteArray("7781c682022000940
```

This is a 201 bytes long response giving a lot of informations back to the terminal. Using the TLV-decoder gives this result (link to the data):

```
77 Response Message Template Format 2
  82 Application Interchange Profile
    2000
  94 Application File Locator (AFL)
    10030600
  57 Track 2 Equivalent Data
    4921828094896752D25022013650000000000F
  9F10 Issuer Application Data
    06040A03A02000
  9F26 Application Cryptogram
    569F946D585CB013
  9F27 Cryptogram Information Data
    80
  9F36 Application Transaction Counter (ATC)
    03A4
  9F4B Signed Dynamic Application Data
    76F770B228B28C0B5A3F2B08C3F2EC3BE6DE27A2708DE06180FDAAEC5482E38CD62D5FE43CF
  9F6C Unknown tag
    1600
```

There are 3 important tags I’m focusing to:

- Application File Locator (AFL)
- Track 2 Equivalent Data
- Application Transaction Counter (ATC)

Application File Locator (AFL)

The **Application File Locator (AFL)** is a pointer to the Credit Card’s internal file directory. Each pointer is a 4 bytes long directory entry with this structure:

```
Application File Locator (AFL)
10 03 06 00
10 - SFI ("directory subfolder")
   03 - first file number to read
      06 - last file number to read
         00 - is a file used during offline authentication ?
```

In total the reader is informed that he can **read 4 files** in total (file numbers 3, 4, 5 and 6). The AFL is a **list** so there can be more blocks of each 4 bytes long to define

more files. The records are read with the **“Read Record” command.**

Track 2 Equivalent Data

This is a leftover from a time where Credit Cards cards wasn’t just “printed” but had a **magnet stripe on the backside.** Those cards got swiped through the reader slot and on the magnet stripe were 2 tracks. To be compliant with older reader software (not hardware, those cards are outdated since a long time) some Credit Cards still contain the data of this track in a tag.

Here is a description from EMV Lab (see tag ‘57’):

```
Track 2 Equivalent Data structure
Primary Account Number
Field Separator (Hex 'D')
Expiration Date (YYMM)
Service Code
Discretionary Data (defined by individual payment systems)
Pad with one Hex 'F' if needed to ensure whole bytes

Complete length: 19 bytes
```

```
Track 2 Equivalent Data
4930005025003985D260920121664081000 00F
4930005025003985 - Primary Account Number
                 D - Field Separator (Hex 'D')
                  2609 - Expiration Date (YYMM)
                      201216 - Service Code
                            6408100000 - Discretionary Data
                                         (defined by individual
                                         payment systems)
                                       F - Pad with one Hex 'F' if needed
                                           to ensure whole bytes
```

If available the “Track 2 Equivalent Data” gives us 2 main data from the tag — the **Primary Account Number (PAN)** that is the **Credit Card Number** and the **Expiration Date** of the card.

> *A note on the published data: all data are from Credit Cards that are out of use since a long time, so e.g.* ***the Credit Card number above is no longer valid and the card is blocked !***

Application Transaction Counter (ATC)

To overcome some possible attack scenarios, an additional monotonically increasing transaction counter (ATC) that is managed by the card is used.

```
Application Transaction Counter (ATC)
03A4 = 0x03A4h = 932 decimal
```

In the end the value is increased by 1 each time a “**Get Processing Options” (“GPO”)** command is sent to the card. But as you can imagine, a 2 bytes long counter has a maximum value of *0xFFFFh* = 65535 decimal. You may think “*how should I ever reach this limit*” but when running a GPO command in a loop it gets fast to bring the counter to the limit. If the counter reaches the limit no more GPO commands are answered by the card and in the end the card is useless.

What is the next action a POS terminal will run ?

Read Record command

To collect more data from the Credit Card the terminal tries to read the data stored in files (“records”). As defined in the “**Application File Locator (AFL)**” list there are 4 files to read. This is the command to

Read Record number 3 in SFI 10:

```
Read Record command for SFI 10 and number 3:
00b2021400
```

I don’t explain the structure of the command as there is some bit shifting in the parameter to set the parameter. The Credit Card responds this data:

```
readRecord response length: 256 data: 7081fb9081f88893cf85a81325ab8da6a4196eb57
70 EMV Proprietary Template
  90 Issuer Public Key Certificate
    8893CF85A81325AB8DA6A4196EB5787291DB7205F61B172B26DEB867DA427F1D0E438E86400
```

The “**Issuer Public Key Certificate**” is an internal data element used for verification of the genuineness of the card.

Read Record number 4 in SFI 10:

```
Read Record number 4 in SFI 10 command:
00b2031400

readRecord response length: 11 data: 70079f3201038f01099000
70 07 -- Record Template (EMV Proprietary)
      9F 32 01 -- Issuer Public Key Exponent
               03 (BINARY)
      8F 01 -- Certification Authority Public Key Index - card
            09 (BINARY)
```

Both data “Issuer Public Key Exponent” and “Certification Authority Public Key Index — card” are internal data to use the certificate data.

Read Record number 5 in SFI 10:

```
Read Record number 5 in SFI 10 command:
00b2051400

readRecord response length: 185 data: 7081b49f4681b047461ffca14b5dfdc209569c8a1
70 81 B4 -- Record Template (EMV Proprietary)
         9F 46 81 B0 -- ICC Public Key Certificate
                     47 46 1F FC A1 4B 5D FD C2 09 56 9C 8A 14 F1 76
                     44 25 1A A3 F4 AB EA 25 12 62 13 4B 92 09 82 F0
                     25 07 41 F9 6F CC B4 08 00 29 30 54 C0 D8 98 24
                     BA 7A C4 4E E7 BA B0 6F A1 57 FC CF 7E 52 D3 C6
                     4B 4D 8A CD 41 B9 77 4B 80 15 19 ED 6F EC 82 7E
                     C2 EC 29 F8 99 11 67 C4 53 77 65 59 A4 A0 6F D9
                     8C 4B 9B D1 54 8A 65 AF 2F 56 00 2A 83 6B DF 9A
                     04 0A 92 53 E6 53 58 4C 92 83 3C 3D 1A A8 E0 8C
                     4D E9 CD A1 02 60 44 F8 0F 39 A9 32 6A 57 49 65
                     98 98 7A 6B 3E 18 A5 F5 6A 8B DE DE 75 28 70 E8
                     79 37 76 DB 9D 32 5C CD 9C 7C A5 DB 33 C2 8F 04 (BINARY)
```

The “ICC Public Key Certificate” is as well an internal element. As you need keys to decrypt the data our online service is not being able to do this for us.

Read Record number 6 in SFI 10:

```
Read Record number 6 in SFI 10 command:
00b2041400

readRecord response length: 240 data: 7081eb9f4681b07e3b33a489fb75a23643407d2eb
70 81 EB -- Record Template (EMV Proprietary)
        9F 46 81 B0 -- ICC Public Key Certificate
                    7E 3B 33 A4 89 FB 75 A2 36 43 40 7D 2E BF 48 A8
                    08 95 71 65 AA 53 8D 68 12 13 D7 14 95 B5 77 08
                    6E 63 A2 4E 84 7E D2 9D 2C EB A4 BB 3B 17 84 36
                    12 21 28 76 07 AC E4 B8 BF CE 09 DD 83 64 D4 70
                    92 93 ED 52 B5 28 62 34 72 FB 61 57 09 4B 12 36
                    75 34 D7 CF 5C 20 B8 10 05 8C 81 7F B8 7C 13 01
                    11 EE 53 C3 85 5F D2 B2 A9 54 49 D0 37 95 54 1E
                    A7 C6 EF 94 2B 0B 06 9B FA 7C AA 5D 0E C6 DB 0E
                    42 8F 18 D0 3A DC F7 F9 2F B7 E5 51 64 03 AD C6
                    29 F3 FF BD 69 00 A1 F3 08 FB E5 D2 8C BA 79 5C
                    6C 62 D7 57 33 33 AB ED 15 AD 00 A4 DA 4B A8 A9 (BINARY)
        9F 47 01 -- ICC Public Key Exponent
                 03 (BINARY)
        5A 08 -- Application Primary Account Number (PAN)
              49 30 00 50 25 00 39 85 (NUMERIC)
        5F 24 03 -- Application Expiration Date
                 26 09 30 (NUMERIC)
        5F 28 02 -- Issuer Country Code
                 02 76 (NUMERIC)
        5F 34 01 -- Application Primary Account Number (PAN) Sequence Number
                 00 (NUMERIC)
        9F 07 02 -- Application Usage Control
                 C0 80 (BINARY)
        9F 4A 01 -- Static Data Authentication Tag List
                 82 (BINARY)
        9F 6E 04 -- Visa Low-Value Payment (VLP) Issuer Authorisation Code
                 20 70 00 00 (BINARY)
        9F 69 07 -- UDOL
                 01 B4 19 C7 27 38 00 (BINARY)
```

Beneath some internal data this record contains the “**Application Primary Account Number (PAN)**” (the Credit Card number) and the “**Application Expiration Date**”. Again the note: **the displayed Credit Card number is outdated.**

Read Record summary

You can read the data as well in the online TLV-decoder, here are the direct links:

- Read Record 3 in SFI 10
- Read Record 4 in SFI 10

- Read Record 5 in SFI 10
- Read Record 6 in SFI 10

At this point we collected and displayed all data that a POS terminal will collect as well. Now we are going to **implement the “question and answer” protocol** for our HCE emulated Credit Card.

Implement the “Question and Answer” Protocol

The next steps are easy to implement as we know all command byte sequences and the responses belonging to it. For this tutorial app I’m using **static byte arrays**, but a “real world” application should be more flexible, e.g. by using a database or a file based approach for more applications on the card or other cards.

To separate from “Basic” service kindly duplicate the HCE CC service class to a file e.g. “HceCcEmulationServiceVisa.java”. As the name says we are emulating a Visa Card with this service.

Before we are proceeding there are 3 steps to go and please, follow this steps to avoid any failure. I’m personally forget them often and wonder why my app is not working as expected…

- in “AndroidManifest.xml” we need to change the binding for the service from Basic to Visa:

```
<service
    android:name=".HceCcEmulationServiceVisa"
    android:exported="true"
    android:permission="android.permission.BIND_NFC_SERVICE">
    <intent-filter>
...
```

- The most important step: **delete your app from your device !** You may think this is not necessary but already running HCE service classes don’t get changed by simply changing the manifest file. Use the “Run app” button in Android Studio to reinstall the app with the changed service class next.
- As mentioned in the “**Troubleshooting**” chapter above, **verify that the HCE CC service on the Payment tab is the default application for processing payment**

commands. In most cases no application is selected any longer and your app will not receive any commands:



Define the commands and responses

I already explained all command and response sequences above, so I’m just showing the static data in the class header:

```
private static final byte[] RESPONSE_OK_SW = hexStringToByteArray("9000");
// Note: the responses does not include the 0x9000h terminator
private static final byte[] SELECT_PPSE_COMMAND = hexStringToByteArray("00a4040
private static final byte[] SELECT_PPSE_RESPONSE = hexStringToByteArray("6f2b84

private static final byte[] SELECT_AID_COMMAND = hexStringToByteArray("00a40400
private static final byte[] SELECT_AID_RESPONSE = hexStringToByteArray("6f5d840

// this needs to be dynamic as the response from the real NFC reader is not pre
private static final byte[] GET_PROCESSING_OPTONS_COMMAND_SHORTED = hexStringTo
// private static final byte[] GET_PROCESSING_OPTONS_COMMAND_READER = hexString
private static final byte[] GET_PROCESSING_OPTONS_RESPONSE = hexStringToByteArr

private static final byte[] READ_FILE_10_03_COMMAND = hexStringToByteArray("00b
private static final byte[] READ_FILE_10_03_RESPONSE = hexStringToByteArray("70

private static final byte[] READ_FILE_10_04_COMMAND = hexStringToByteArray("00b
private static final byte[] READ_FILE_10_04_RESPONSE = hexStringToByteArray("70

private static final byte[] READ_FILE_10_05_COMMAND = hexStringToByteArray("00b
private static final byte[] READ_FILE_10_05_RESPONSE = hexStringToByteArray("70

private static final byte[] READ_FILE_10_06_COMMAND = hexStringToByteArray("00b
private static final byte[] READ_FILE_10_06_RESPONSE = hexStringToByteArray("70
```

As the POS terminal will send a “GET_PROCESSING_OPTONS_COMMAND” with data belonging to a transaction and his environment we cannot compare the full

command — it won’t match. So instead I’m using just the first 2 bytes of the command to identify the command (*0x80a8h*).

Logic in processCommandApdu method

To keep the tutorial app as short and easy to understand I did not implement any indicators to automatically select between different cards or so. The service will emulate exact one Visa card with one application, that’s it. For that reason the complete logic is very simple:

```
// step 01
if (Arrays.equals(commandApdu, SELECT_PPSE_COMMAND)) {
    // step 01 selecting the PPSE
    sendMessageToActivity("step 01 Select PPSE Command ", bytesToHexNpe(commanc
    sendMessageToActivity("step 01 Select PPSE Response", bytesToHexNpe(SELECT_
    return concatenateByteArrays(SELECT_PPSE_RESPONSE, RESPONSE_OK_SW);
}

// step 02
if (Arrays.equals(commandApdu, SELECT_AID_COMMAND)) {
    // step 02 selecting the AID
    sendMessageToActivity("step 02 Select AID Command ", bytesToHexNpe(commandA
    sendMessageToActivity("step 02 Select AID Response", bytesToHexNpe(SELECT_A
    return concatenateByteArrays(SELECT_AID_RESPONSE, RESPONSE_OK_SW);
}

// step 03
// dynamic - just compare the beginning of the commandApdu with the sample
byte[] shortedCommandApdu = Arrays.copyOf(commandApdu, 2);
if (Arrays.equals(shortedCommandApdu, GET_PROCESSING_OPTONS_COMMAND_SHORTED)) {
    sendMessageToActivity("step 03 Get Processing Options (GPO) Command", bytes
    sendMessageToActivity("step 03 Get Processing Options (GPO) Response", byte
    return concatenateByteArrays(GET_PROCESSING_OPTONS_RESPONSE, RESPONSE_OK_SW
}

// step 04
if (Arrays.equals(commandApdu, READ_FILE_10_03_COMMAND)) {
    // step 04 Read File 10/03 Command
    sendMessageToActivity("step 04 Read File 10/03 Command", bytesToHexNpe(comm
    sendMessageToActivity("step 04 Read File 10/03 Response", bytesToHexNpe(REA
    return concatenateByteArrays(READ_FILE_10_03_RESPONSE, RESPONSE_OK_SW);
}

// step 05
if (Arrays.equals(commandApdu, READ_FILE_10_04_COMMAND)) {
    // step 05 Read File 10/04 Command
    sendMessageToActivity("step 05 Read File 10/04 Command", bytesToHexNpe(comm
    sendMessageToActivity("step 05 Read File 10/05 Response", bytesToHexNpe(REA
    return concatenateByteArrays(READ_FILE_10_04_RESPONSE, RESPONSE_OK_SW);
```

```
    }

    // step 06
    if (Arrays.equals(commandApdu, READ_FILE_10_05_COMMAND)) {
        // step 06 Read File 10/05 Command
        sendMessageToActivity("step 06 Read File 10/05 Command", bytesToHexNpe(comm
        sendMessageToActivity("step 06 Read File 10/05 Response", bytesToHexNpe(REA
        return concatenateByteArrays(READ_FILE_10_05_RESPONSE, RESPONSE_OK_SW);
    }

    // step 07
    if (Arrays.equals(commandApdu, READ_FILE_10_06_COMMAND)) {
        // step 07 Read File 10/06 Command
        sendMessageToActivity("step 08 Read File 10/06 Command", bytesToHexNpe(comm
        sendMessageToActivity("step 08 Read File 10/06 Response", bytesToHexNpe(REA
        return concatenateByteArrays(READ_FILE_10_06_RESPONSE, RESPONSE_OK_SW);
    }

    // in any other case response an 'OK'
    return RESPONSE_OK_SW;
```

A note on the “concatenateByteArrays” method: I personally don’t like to mix different data in one byte array, for that reason the responses don’t include the terminating “*0x9000h*” at the end. For that reason I’m concatenating the “pure” response with the “OK” terminator:

```
in Utils.java class:
public static byte[] concatenateByteArrays(byte[] dataA, byte[] dataB) {
    byte[] concatenated = new byte[dataA.length + dataB.length];
    for (int i = 0; i < dataA.length; i++) {
        concatenated[i] = dataA[i];
    }
    for (int i = 0; i < dataB.length; i++) {
        concatenated[dataA.length + i] = dataB[i];
    }
    return concatenated;
}
```

Now we are ready to build and run the app. Tap the other Android device running the “Talk to your Credit Card” app against your HCE device and see what’s happening. After a successful connection you see a lot of data coming up:



Display data from the HCE service class in the MainActivity

I did mention before that the HCE service class is running as a **background service**. For that reason there is no direct connection between the service and the class that started the service. There are different ways to send the data from the service to an activity and I’m using a **Broadcast Receiver** to do the job. In the MainActivity the receiver is registered and in the HCE service class I’m using an **Intent** to send my

data through Android’s message system to the MainActivity. The registered Broadcast Receiver grabs the messages out of the pool and displays the data in the log file. There is no other interaction implemented — if you should need one it’s up to you to do so.

Automate a data import from an existing card to emulate

Of course, it can be a little bit annoying to “copy & paste” the commands and responses of a tag into the app for your own Credit Card and yes, there is a way to do this task “semi-automatic”. I’ve done this for my own purpose by collecting the data in a “model” class and exporting the data in a JSON file within the “Talk to your Credit Card” app. The file is then imported to this app and emulates the complete tag with some clicks. The bad news is — this app isn’t available in the code of this app because it should be as simple as possible, sorry.

Emulating a MasterCard

Beneath **Visa** the Credit Cards issued by **MasterCard** are the most common Credit Cards in the world. As they follow the EMV Standard they can get implemented exact the same way we are doing for a Visa Card. I created a separated class for the MasterCard emulation (“HceCcEmulationServiceMastercard.java”). Please be aware that this class and its “processCommandApdu” method is never triggered by the Android OS if you forgot to add the AID “a0000000041010” to your “apduservice.xml & strings.xml” files.

Please don’t forget to **delete the app** on your Android device as the service class is changed from ...Visa to ...MasterCard !

I don’t explain each command or response as it is equal to the Visa one (yes, some other file numbers are used) except the “Select AID” response and “Get Processing Options” command as they differ from the Visa example.

Select Application (“Select AID”) response

The response of the “Select AID” command on a MasterCard looks different, here is decoded data (link to the data):

```
6F File Control Information (FCI) Template
  84 Dedicated File (DF) Name
    A0000000041010
  A5 File Control Information (FCI) Proprietary Template
    50 Application Label
```

```
    D e b i t M a s t e r C a r d
  9F12 Application Preferred Name
    D e b i t M a s t e r C a r d
  87 Application Priority Indicator
    01
  9F11 Issuer Code Table Index
    01
  5F2D Language Preference
    d e e n
  BF0C File Control Information (FCI) Issuer Discretionary Data
    9F0A Unknown tag
      00010101
    9F6E Unknown tag
      02800000303000
```

This response does not include the “**Processing Options Data Object List**” (PDOL) tag, so card is not requesting to get those data elements. But the next mandatory step for the POS Terminal is to request the “Processing Options” — how to run this command with any requested data ?

Get Processing Options (GPO) command

The answer is easy — we provide an “empty” PDOL list:

```
byte[] GET_PROCESSING_OPTONS_COMMAND = hexStringToByteArray("80a8000002830000")
Get Processing Options (GPO) command
80a8000002830000
80 - Command
  A8 - Instruction Code
    0000 - Sub command parameter P1 and P2
        02 - Length of following data (0x02h = 2 decimal) including '83' and '0
          83 - static identifier indicating that no PDOL will follow
            00 Length of the PDOL data (0x00h = 0 decimal)
              00 - Finalisation
```

B.t.w. this won’t work on a Visa Card as they usually expect PDOL data.

The next steps are as shown in the Visa Card chapter, for a full response see the MasterCard log file example in the repository.

Whereas the Visa Card exposed the PAN and Expire date in the “Get Processing Options (GPO)” response my MasterCard gives this information in several files only.

At this point our tutorial ends as our app is fully responding to all commands the POS Terminal is requesting (maybe there are additional commands by the terminal but I didn’t recocnize them). The terminal is being able to process the transaction by sending the data to it’s processor. Depending on the parameter the show owner defined the card holder is required to enter a PIN or show an additional identification.

Summary

The Host Based Card Emulation (HCE) of a Credit Card (EMV card) is not too difficult technology when implemented correctly. There are just some commands and responses to implement (“Questions & Answers”) and the virtual Credit Card is ready to use. We learned how to analyze the commands and responses and understand what data is processed between a OS Terminal and the (HCE) Credit Card.

The sample app is working on all NFC & HCE enabled Android smartphones with a SDK version starting version 21 (Android 5).

Source code of the app

Thats it for this tutorial, you find the **source code for the complete app** in my GitHub repository https://github.com/AndroidCrypto/Android_HCE_Emulate_A_CreditCard/.

If you want to use a **pre compiled app** I’m providing a “**ready to use app**”, compiled in debug mode in the subfolder “sampleapp”. This app will emulate a Visa Card.

Happy coding !

Android + Nfc + Host Based Card Emulation + Nfc Reader +

Programming +



Follow

Written by AndroidCrypto

643 followers · 4 following